

vehicle/telematics
↓
Python Subscriber
↓
SQLite
↓
Google Sheets

Recommended Architecture

Vehicle Simulator
↓
Virtual CAN
↓
ECU Receiver
↓
MQTT Broker
↓
Python Telematics Service
├── Google Sheets (main)
└── SQLite (POC)

Developed a Linux-based Automotive Telematics Control Unit simulator featuring CAN communication, MQTT-based telemetry transmission, SQLite data persistence, and cloud logging to Google Sheets for remote vehicle monitoring.

- ✓ Single MQTT topic : `vehicle/telematics`
- ✓ JSON payload using : `nlohmann/json`
- ✓ CAN simulation unchanged
- ✓ Console output retained
- ✓ JSON payload printed to terminal

Install JSON Library

Run:

None

```
sudo apt update  
sudo apt install nlohmann-json3-dev
```

Verify:

None

```
ls /usr/include/nlohmann/json.hpp
```

Check 1 : Verify the JSON library is installed

Run:

None

```
dpkg -s nlohmann-json3-dev | grep Version
```

CODE :

None

```
#include <iostream>  
#include <cstdlib>  
#include <ctime>  
#include <unistd.h>  
#include <cstring>  
  
#include <mosquitto.h>  
  
#include <nlohmann/json.hpp>  
  
// CAN headers  
#include <linux/can.h>  
#include <linux/can/raw.h>  
  
#include <sys/socket.h>
```

```
#include <sys/ioctl.h>

#include <net/if.h>

using json = nlohmann::json;

struct GPSTData
{
    double latitude;
    double longitude;
};

struct VehicleData
{
    int speed;
    int rpm;
    int fuel;
    int engineTemp;
    GPSTData gps;
};

int main()
{
    srand(time(NULL));

    VehicleData vehicle;

    vehicle.gps.latitude = 13.0827;
    vehicle.gps.longitude = 80.2707;

    // MQTT Initialization
    mosquitto_lib_init();

    struct mosquitto *mosq =
        mosquitto_new("TCU_Client", true, NULL);
```

```

if(!mosq)
{
    std::cerr << "Failed to create MQTT client\n";
    return 1;
}

if(mosquitto_connect(
    mosq,
    "localhost",
    1883,
    60) != MOSQ_ERR_SUCCESS)
{
    std::cerr << "Failed to connect to broker\n";

    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();

    return 1;
}

std::cout << "Connected to MQTT Broker\n";

// -----
// CAN Socket Initialization
// -----

int can_socket;

struct sockaddr_can addr;
struct ifreq ifr;

can_socket =
    socket(PF_CAN,
           SOCK_RAW,
           CAN_RAW);

```

```

strcpy(ifr.ifr_name, "vcan0");

ioctl(can_socket,
       SIOCGIFINDEX,
       &ifr);

addr.can_family = AF_CAN;
addr.can_ifindex = ifr.ifr_ifindex;

bind(can_socket,
     (struct sockaddr *)&addr,
     sizeof(addr));

std::cout << "Connected to vcan0\n";

while(true)
{
    // -----
    // Generate Vehicle Data
    // -----

    vehicle.speed = rand() % 120;
    vehicle.rpm = rand() % 6000;
    vehicle.fuel = rand() % 100;
    vehicle.engineTemp =
        80 + rand() % 21;

    // -----
    // CAN Frames
    // -----

    struct can_frame speedFrame;
    struct can_frame rpmFrame;
    struct can_frame tempFrame;

    // Speed Frame (0x100)

```

```
speedFrame.can_id = 0x100;
speedFrame.can_dlc = 1;

speedFrame.data[0] =
    vehicle.speed;

// RPM Frame (0x101)

rpmFrame.can_id = 0x101;
rpmFrame.can_dlc = 2;

rpmFrame.data[0] =
    (vehicle.rpm >> 8) & 0xFF;

rpmFrame.data[1] =
    vehicle.rpm & 0xFF;

// Temperature Frame (0x102)

tempFrame.can_id = 0x102;
tempFrame.can_dlc = 1;

tempFrame.data[0] =
    vehicle.engineTemp;

// -----
// Send CAN Frames
// -----

write(can_socket,
      &speedFrame,
      sizeof(speedFrame));

write(can_socket,
      &rpmFrame,
```

```
        sizeof(rpmFrame));

write(can_socket,
      &tempFrame,
      sizeof(tempFrame));

// -----
// GPS Simulation
// -----

vehicle.gps.latitude += 0.0001;
vehicle.gps.longitude += 0.0001;

// -----
// Create JSON Telemetry Packet
// -----

json telemetry;

telemetry["latitude"] =
    vehicle.gps.latitude;

telemetry["longitude"] =
    vehicle.gps.longitude;

telemetry["speed"] =
    vehicle.speed;

telemetry["rpm"] =
    vehicle.rpm;

telemetry["fuel"] =
    vehicle.fuel;

telemetry["engineTemp"] =
    vehicle.engineTemp;
```

```

std::string payload =
    telemetry.dump();

// -----
// Publish MQTT
// -----

mosquitto_publish(
    mosq,
    NULL,
    "vehicle/telematics",
    payload.length(),
    payload.c_str(),
    0,
    false);

// -----
// Console Output
// -----

std::cout
    << "\n=====\n";

std::cout
    << "Vehicle Telemetry\n";

std::cout
    << "=====\n";

std::cout
    << "Latitude      : "
    << vehicle.gps.latitude
    << std::endl;

std::cout

```



```
        << "Longitude      : "  
        << vehicle.gps.longitude  
        << std::endl;  
  
    std::cout  
        << "Speed          : "  
        << vehicle.speed  
        << " km/h"  
        << std::endl;  
  
    std::cout  
        << "RPM            : "  
        << vehicle.rpm  
        << std::endl;  
  
    std::cout  
        << "Fuel            : "  
        << vehicle.fuel  
        << "%"  
        << std::endl;  
  
    std::cout  
        << "Engine Temp   : "  
        << vehicle.engineTemp  
        << " C"  
        << std::endl;  
  
    std::cout  
        << "\nJSON Payload:\n"  
        << payload  
        << std::endl;  
  
    sleep(1);  
}  
  
// Cleanup
```

```
    mosquitto_disconnect(mosq);  
    mosquitto_destroy(mosq);  
    mosquitto_lib_cleanup();  
  
    return 0;  
}
```

Compile

None

```
sudo apt install nlohmann-json3-dev
```

```
g++ tcu_1publisher_mqttCAN.cpp \  
-o tcu_1publisher_mqttCAN \  
-lmosquitto \  
-std=c++17
```

Test MQTT

Terminal 2

None

```
mosquitto_sub -h localhost -t vehicle/telematics
```

```
aniruddhan@raspberrypi:~$ mosquitto_sub -h localhost -t vehicle/telematics
{"engineTemp":86,"fuel":50,"latitude":13.0828,"longitude":80.27080000000001,"rpm":1852,"speed":0}
{"engineTemp":100,"fuel":2,"latitude":13.0829,"longitude":80.27090000000001,"rpm":263,"speed":5}
{"engineTemp":96,"fuel":10,"latitude":13.083,"longitude":80.27100000000002,"rpm":592,"speed":42}
{"engineTemp":90,"fuel":9,"latitude":13.0831,"longitude":80.27110000000002,"rpm":3767,"speed":60}
{"engineTemp":93,"fuel":87,"latitude":13.0832,"longitude":80.27120000000002,"rpm":4664,"speed":0}
{"engineTemp":92,"fuel":48,"latitude":13.0833,"longitude":80.27130000000002,"rpm":3522,"speed":5}
{"engineTemp":98,"fuel":55,"latitude":13.0834,"longitude":80.27140000000003,"rpm":3019,"speed":89}
{"engineTemp":98,"fuel":2,"latitude":13.083499999999999,"longitude":80.27150000000003,"rpm":1727,"speed":71}
{"engineTemp":83,"fuel":67,"latitude":13.083599999999999,"longitude":80.27160000000003,"rpm":752,"speed":91}
{"engineTemp":98,"fuel":58,"latitude":13.083699999999999,"longitude":80.27170000000004,"rpm":1269,"speed":56}
{"engineTemp":92,"fuel":34,"latitude":13.083799999999998,"longitude":80.27180000000004,"rpm":4268,"speed":62}
{"engineTemp":81,"fuel":46,"latitude":13.083899999999998,"longitude":80.27190000000004,"rpm":4743,"speed":107}
{"engineTemp":98,"fuel":27,"latitude":13.083999999999998,"longitude":80.27200000000005,"rpm":4885,"speed":39}
{"engineTemp":99,"fuel":5,"latitude":13.084099999999998,"longitude":80.27210000000005,"rpm":5527,"speed":119}
```

Terminal 1

None

```
./tcu_1publisher_mqttCAN
```

```
aniruddhan@raspberrypi:~$ ./tcu_1publisher_mqttCAN
Connected to MQTT Broker
Connected to vcan0

=====
Vehicle Telemetry
=====
Latitude    : 13.0828
Longitude   : 80.2708
Speed       : 0 km/h
RPM          : 1852
Fuel         : 50%
Engine Temp  : 86 C

JSON Payload:
{"engineTemp":86,"fuel":50,"latitude":13.0828,"longitude":80.27080000000001,"rpm":1852,"speed":0}

=====
Vehicle Telemetry
=====
Latitude    : 13.0829
Longitude   : 80.2709
Speed       : 5 km/h
RPM          : 263
Fuel         : 2%
Engine Temp  : 100 C

JSON Payload:
{"engineTemp":100,"fuel":2,"latitude":13.0829,"longitude":80.27090000000001,"rpm":263,"speed":5}
```

The JSON MQTT telemetry pipeline is now working perfectly.

Your output confirms:

```
None
Vehicle Simulator
    ↓
CAN Frames
    ↓
MQTT Publisher
    ↓
Mosquitto Broker
    ↓
JSON Telemetry
```

is functioning correctly.

=====

CHANGES MADE TO THE NEW CODE

1. Add JSON Header

After:

None

```
#include <mosquitto.h>
```

add:

None

```
#include <nlohmann/json.hpp>
```

and after the includes add:

None

```
using json = nlohmann::json;
```

2. Remove Old MQTT Message Creation

Delete this entire block:

None

```
char locationMsg[100];

char statusMsg[100];

char diagnosticMsg[100];

snprintf(locationMsg,
          sizeof(locationMsg),
          "Latitude=%.4f Longitude=%.4f",
          vehicle.gps.latitude,
          vehicle.gps.longitude);
```

```
snprintf(statusMsg,
          sizeof(statusMsg),
          "Speed=%d Fuel=%d",
          vehicle.speed,
          vehicle.fuel);

snprintf(diagnosticMsg,
          sizeof(diagnosticMsg),
          "RPM=%d EngineTemp=%d",
          vehicle.rpm,
          vehicle.engineTemp);
```

3. Remove Old MQTT Publish Calls

Delete this entire section:

```
None
// Publish Location

mosquitto_publish(
    mosq,
    NULL,
```

```
        "vehicle/location",  
        strlen(locationMsg),  
        locationMsg,  
        0,  
        false);  
  
// Publish Status  
mosquitto_publish(  
    mosq,  
    NULL,  
    "vehicle/status",  
    strlen(statusMsg),  
    statusMsg,  
    0,  
    false);  
  
// Publish Diagnostics  
mosquitto_publish(  
    mosq,  
    NULL,  
    "vehicle/diagnostics",
```

```
strlen(diagnosticMsg),  
diagnosticMsg,  
0,  
false);
```

4. Replace With JSON Telemetry Block

Insert this in its place:

None

```
json telemetry;  
  
telemetry["latitude"] =  
    vehicle.gps.latitude;  
  
telemetry["longitude"] =  
    vehicle.gps.longitude;  
  
telemetry["speed"] =  
    vehicle.speed;  
  
telemetry["rpm"] =
```



```
    vehicle.rpm;

    telemetry["fuel"] =
        vehicle.fuel;

    telemetry["engineTemp"] =
        vehicle.engineTemp;

    std::string payload =
        telemetry.dump();

    mosquitto_publish(
        mosq,
        NULL,
        "vehicle/telematics",
        payload.length(),
        payload.c_str(),
        0,
        false
    );
```

5. Add JSON Print To Console

After:

None

```
std::cout << "Engine Temp  : "  
  
    << vehicle.engineTemp << " C" << std::endl;
```

add:

None

```
std::cout << "\nJSON Payload:\n"  
  
    << payload  
  
    << std::endl;
```

6. Remove Old Subscribers

You no longer need:

None

```
mosquitto_sub -h localhost -t vehicle/location  
  
mosquitto_sub -h localhost -t vehicle/status  
  
mosquitto_sub -h localhost -t vehicle/diagnostics
```

Replace them with:

None

```
mosquitto_sub -h localhost -t vehicle/telematics
```

DATA LOGGING 1-PUBLISHER AND 1-SUBSCRIBER TCU CODE

New Database Structure

None

```
vehicle_telematics_V2.db
```

Create new logger:

None

```
nano vehicle_logger_json.py
```

Paste:

None

```
import sqlite3
import json
import paho.mqtt.client as mqtt

# -----
# Database Setup
# -----

conn = sqlite3.connect(
    "vehicle_telematics_v2.db"
```

```

)

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS telematics(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    speed INTEGER,
    rpm INTEGER,
    fuel INTEGER,
    engine_temp INTEGER,
    latitude REAL,
    longitude REAL
)
""")

conn.commit()

# -----
# MQTT Callback
# -----

def on_message(client, userdata, msg):

    payload = msg.payload.decode()

    data = json.loads(payload)

    speed = data["speed"]
    rpm = data["rpm"]
    fuel = data["fuel"]
    engine_temp = data["engineTemp"]

```

```

latitude = data["latitude"]
longitude = data["longitude"]

cursor.execute(
    """
        INSERT INTO telematics(
            speed,
            rpm,
            fuel,
            engine_temp,
            latitude,
            longitude
        )
        VALUES(?, ?, ?, ?, ?, ?, ?)
    """,
    (
        speed,
        rpm,
        fuel,
        engine_temp,
        latitude,
        longitude
    )
)

conn.commit()

print(
    f"Logged: "
    f"Speed={speed}, "
    f"RPM={rpm}, "
    f"Fuel={fuel}"
)

```

```
# -----  
# MQTT Setup  
# -----  
  
client = mqtt.Client()  
  
client.on_message = on_message  
  
client.connect(  
    "localhost",  
    1883,  
    60  
)  
  
client.subscribe(  
    "vehicle/telematics"  
)  
  
print(  
    "JSON Telematics Logger Started..."  
)  
  
client.loop_forever()
```

Run Logger

Terminal 1

None

```
python3 vehicle_logger_json.py
```

```

aniruddhan@raspberrypi:~ $ python3 vehicle_logger_json.py
/home/aniruddhan/vehicle_logger_json.py:82: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client()
JSON Telematics Logger Started...
Logged: Speed=84, RPM=5523, Fuel=17
Logged: Speed=57, RPM=459, Fuel=75
Logged: Speed=3, RPM=1233, Fuel=0
Logged: Speed=0, RPM=5100, Fuel=33
Logged: Speed=74, RPM=4739, Fuel=73
Logged: Speed=99, RPM=494, Fuel=39
Logged: Speed=96, RPM=2465, Fuel=44
Logged: Speed=85, RPM=1621, Fuel=68
Logged: Speed=64, RPM=737, Fuel=47
Logged: Speed=116, RPM=3974, Fuel=23
Logged: Speed=40, RPM=5175, Fuel=98
Logged: Speed=75, RPM=331, Fuel=65
Logged: Speed=23, RPM=890, Fuel=54
Logged: Speed=64, RPM=1793, Fuel=41
Logged: Speed=59, RPM=538, Fuel=71
Logged: Speed=119, RPM=4192, Fuel=18

```

Terminal 2

None

`./tcu_1publisher_mqttCAN`

```

aniruddhan@raspberrypi:~ $ ./tcu_1publisher_mqttCAN
Connected to MQTT Broker
Connected to vcan0

=====
Vehicle Telemetry
=====
Latitude      : 13.0828
Longitude     : 80.2708
Speed        : 84 km/h
RPM          : 5523
Fuel         : 17%
Engine Temp  : 94 C

JSON Payload:
{"engineTemp":94,"fuel":17,"latitude":13.0828,"longitude":80.27080000000001,"rpm":5523,"speed":84}

=====
Vehicle Telemetry
=====
Latitude      : 13.0829
Longitude     : 80.2709
Speed        : 57 km/h
RPM          : 459
Fuel         : 75%
Engine Temp  : 99 C

JSON Payload:
{"engineTemp":99,"fuel":75,"latitude":13.0829,"longitude":80.27090000000001,"rpm":459,"speed":57}

```

Terminal 3 : Verify Database

Open SQLite:

```
None  
sqlite3 vehicle telematics_v2.db
```

For Show table type:

```
None  
.tables
```

Expected:

```
None  
telematics
```

To Check rows:

```
None  
SELECT * FROM telematics LIMIT 5;
```

```
aniruddhan@raspberrypi:~ $ sqlite3 vehicle telematics_v2.db  
SQLite version 3.40.1 2022-12-28 14:03:47  
Enter ".help" for usage hints.  
sqlite> .tables  
telematics  
sqlite> SELECT * FROM telematics LIMIT 6;  
1|2026-06-13 07:27:32|84|5523|17|94|13.0828|80.2708  
2|2026-06-13 07:27:33|57|459|75|99|13.0829|80.2709  
3|2026-06-13 07:27:34|3|1233|0|100|13.083|80.271  
4|2026-06-13 07:27:35|0|5100|33|86|13.0831|80.2711  
5|2026-06-13 07:27:36|74|4739|73|100|13.0832|80.2712  
6|2026-06-13 07:27:37|99|494|39|95|13.0833|80.2713  
sqlite> █
```


The output that can be observed from the image above:

None

```
1|2026-06-13 07:27:32|84|5523|17|94|13.0828|80.2708
2|2026-06-13 07:27:33|57|459|75|99|13.0829|80.2709
3|2026-06-13 07:27:34|3|1233|0|100|13.083|80.271
```

confirms that:

MQTT JSON Parsing

None

```
{
  "speed":84,
  "rpm":5523,
  "fuel":17,
  "engineTemp":94,
  "latitude":13.0828,
  "longitude":80.2708
}
```

↓

SQLite Insert

None

```
INSERT INTO telematics(...)
```

↓

Database Storage

None

```
speed
rpm
fuel
engine_temp
```

```
latitude
longitude
```

is working correctly.

```
=====
=====
```

NOTE:

Developed a Linux-based Automotive Telematics Control Unit (TCU) simulator featuring SocketCAN communication, MQTT-based JSON telemetry transmission, and SQLite-based vehicle data persistence for real-time telemetry logging.